STM32基础 - 电源管理

课程目标

1. 掌握电源管理的睡眠模式（Sleep）、停机模式（Stop）和待机模式（Standby）

课程试验

1. 利用STM32完成电源管理

课堂引入

为了降低设备功耗，在空闲时可以进行睡眠、停机、待机。

一、PWR简介

•PWR（Power Control）电源控制

•PWR负责管理STM32内部的电源供电部分，可以实现可编程电压监测器和低功耗模式的功能

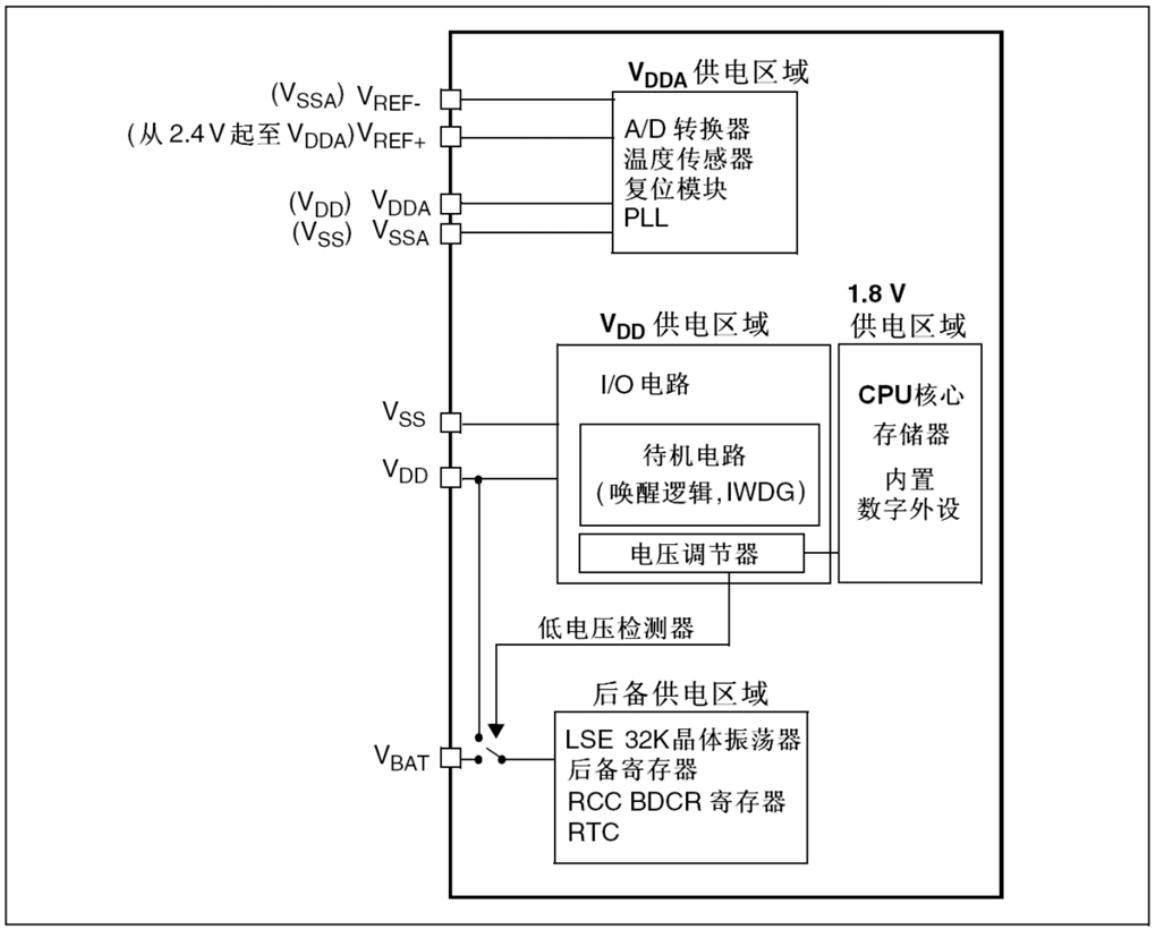
•可编程电压监测器（PVD）可以监控VDD电源电压，当VDD下降到PVD阀值以下或上升到PVD阀值之上时，PVD会触发中断，用于执行紧急关闭任务

•低功耗模式包括睡眠模式（Sleep）、停机模式（Stop）和待机模式（Standby），可在系统空闲时，降低STM32的功耗，延长设备使用时间

二、框图

1、电源框图

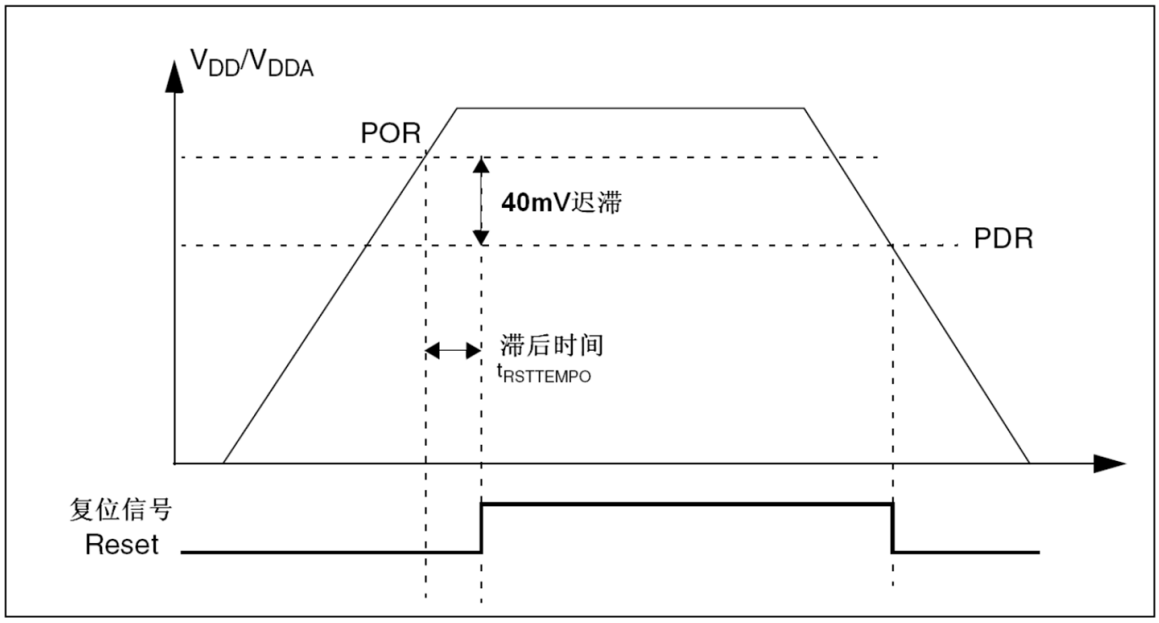
图4　　电源框图



V_{DDA}和V_{SSA}必须分别联到V_{DD}和V_{SS}。

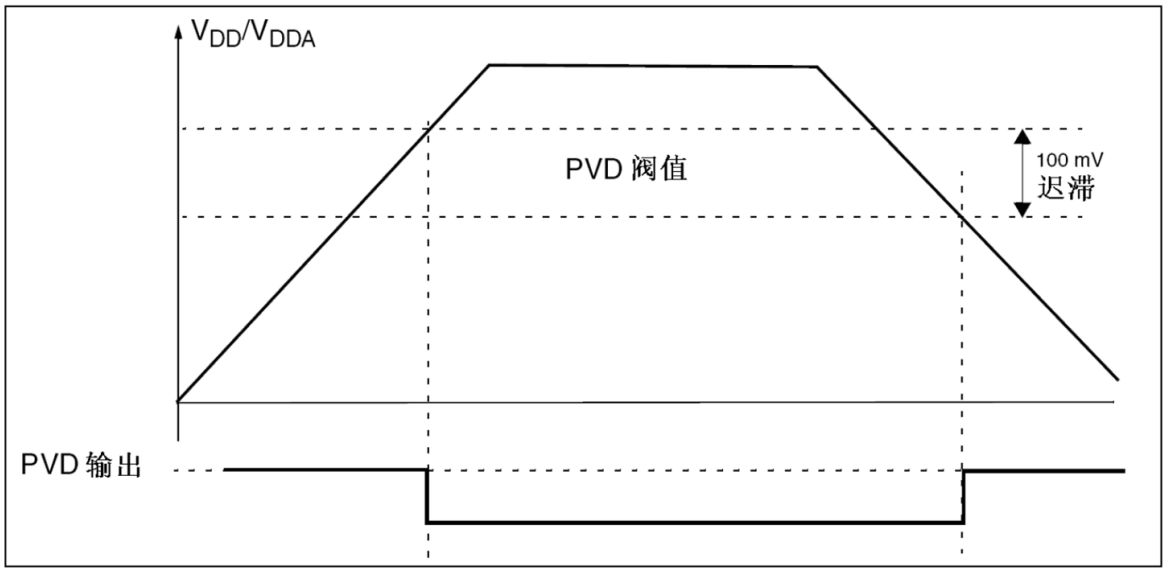
2、上电复位和掉电复位

图5　　上电复位和掉电复位的波形图



3、可编程电压监测器

图6　　PVD的门限



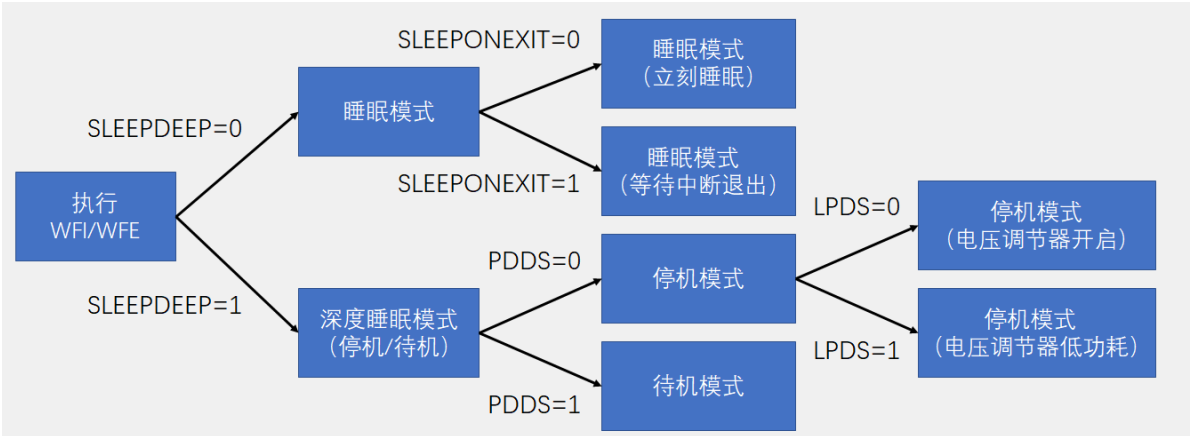
4、低功耗模式

表8　　低功耗模式一览

模式	进入	唤醒	对1.8V区域时钟的影响	对V_{DD}区域时钟的影响	电压调节器
睡眠(SLEEP-NOW或SLEEP-ON-EXIT)	WFI	任一中断	CPU时钟关，对其他时钟和ADC时钟无影响	无	开
	WFE	唤醒事件			
停机	PDDS和LPDS位+SLEEPDEEP位+WFI或WFE	任一外部中断(在外部中断寄存器中设置)	关闭所有1.8V区域的时钟	HSI 和HSE的振荡器关闭	开启或处于低功耗模式(依据电源控制寄存器(PWR_CR)的设定)
待机	PDDS位+SLEEPDEEP位+WFI或WFE	WKUP引脚的上升沿、RTC闹钟事件、NRST引脚上的外部复位、IWDG复位			关

三、模式选择

执行WFI（Wait For Interrupt）或者WFE（Wait For Event）指令后，STM32进入低功耗模式



四、睡眠模式

执行完WFI/WFE指令后，STM32进入睡眠模式，程序暂停运行，唤醒后程序从暂停的地方继续运行

SLEEPONEXIT位决定STM32执行完WFI或WFE后，是立刻进入睡眠，还是等STM32从最低优先级的中断处理程序中退出时进入睡眠

在睡眠模式下，所有的I/O引脚都保持它们在运行模式时的状态

WFI指令进入睡眠模式，可被任意一个NVIC响应的中断唤醒

WFE指令进入睡眠模式，可被唤醒事件唤醒

五、停止模式

执行完WFI/WFE指令后，STM32进入停止模式，程序暂停运行，唤醒后程序从暂停的地方继续运行

1.8V供电区域的所有时钟都被停止，PLL、HSI和HSE被禁止，SRAM和寄存器内容被保留下来

在停止模式下，所有的I/O引脚都保持它们在运行模式时的状态

当一个中断或唤醒事件导致退出停止模式时，HSI被选为系统时钟

当电压调节器处于低功耗模式下，系统从停止模式退出时，会有一段额外的启动延时

WFI指令进入停止模式，可被任意一个EXTI中断唤醒

WFE指令进入停止模式，可被任意一个EXTI事件唤醒

六、待机模式

执行完WFI/WFE指令后，STM32进入待机模式，唤醒后程序从头开始运行

整个1.8V供电区域被断电，PLL、HSI和HSE也被断电，SRAM和寄存器内容丢失，只有备份的寄存器和待机电路维持供电

在待机模式下，所有的I/O引脚变为高阻态（浮空输入）

WKUP引脚的上升沿、RTC闹钟事件的上升沿、NRST引脚上外部复位、IWDG复位退出待机模式

七、实验

1、降低系统时钟实验

（1）文档介绍

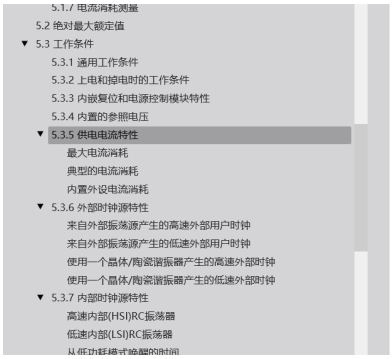
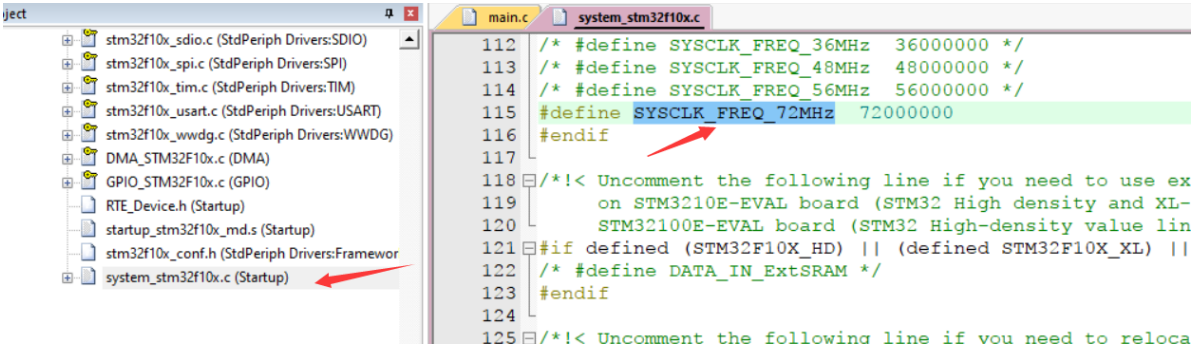


表13 运行模式下的最大电流消耗，数据处理代码从内部闪存中运行

符号	参数	条件	f_{HCLK}	最大值(1)		单位
				T_A = 85°C	T_A = 105°C	
I_{DD}	运行模式下的供应电流	外部时钟(2)，使能所有外设	72MHz	50	50.3	mA
			48MHz	36.1	36.2	
			36MHz	28.6	28.7	
			24MHz	19.9	20.1	
			16MHz	14.7	14.9	
			8MHz	8.6	8.9	
		外部时钟(2)，关闭所有外设	72MHz	32.8	32.9	
			48MHz	24.4	24.5	
			36MHz	19.8	19.9	
			24MHz	13.9	14.2	
			16MHz	10.7	11	
			8MHz	6.8	7.1	

1. 由综合评估得出，不在生产中测试。
2. 外部时钟为8MHz，当f_{HCLK}>8MHz时启用PLL。

（2）降低主频设置



（3）使用代码可以检查

> OLED_ShowNum(1,1,SystemCoreClock,10);

2、睡眠模式

实验目的：

进入睡眠模式后只有cpu在休息，外设还在继续运行。

实验过程：

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "CountSensor.h"

int main(void)
{
    OLED_Init();
    CountSensor_Init();

    while(1)
    {

        OLED_ShowNum(1,1,CountSernsor_Get_Count(),5);
        Delay_ms(1000);
        OLED_Clear();
        Delay_ms(1000);
        //进入睡眠模式:只有cpu在休息，外设在继续运行。
        __WFI();
    }

}
```

3、停机模式

实现目的

中断才能唤醒设备

实验过程

```c
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "CountSensor.h"

int main(void)
{
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR,ENABLE); // 开启电源管理外设时钟

    OLED_Init();
    CountSensor_Init();

    while(1)
    {

        OLED_ShowNum(1,1,CountSernsor_Get_Count(),5);
        OLED_ShowNum(2,1,SystemCoreClock,10);
        Delay_ms(1000);
        OLED_Clear();
        Delay_ms(1000);
        //电压调节器进入低功耗，PWR_Regulator_LowPower: STOP mode with regulator in low
power mode
        PWR_EnterSTOPMode(PWR_Regulator_LowPower,PWR_STOPEntry_WFI);//进入停止模式,中断进入

        SystemInit();//恢复时钟

    }

}
```

4、待机模式

实验目的

待机模式唤醒后，程序会重头开始运行

实验过程

```c
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "CountSensor.h"
#include "MyRTC.h"



int main(void)
{
    /*模块初始化*/
    OLED_Init();        //OLED初始化
    MyRTC_Init();       //RTC初始化
```

```
    /*开启时钟*/
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);      //开启PWR的时钟
                                                             //停止模式和待机模式一定要记得开
启

    /*显示静态字符串*/
    OLED_ShowString(1, 1, "CNT :");
    OLED_ShowString(2, 1, "ALR :");
    OLED_ShowString(3, 1, "ALRF:");

    /*使能WKUP引脚*/
    PWR_WakeUpPinCmd(ENABLE);                          //使能位于PA0的WKUP引脚，WKUP引脚上升沿
唤醒待机模式

    /*设定闹钟*/
    uint32_t Alarm = RTC_GetCounter() + 10;          //闹钟为唤醒后当前时间的后10s
    RTC_SetAlarm(Alarm);                             //写入闹钟值到RTC的ALR寄存器
    OLED_ShowNum(2, 6, Alarm, 10);                   //显示闹钟值

    //简单的做个加法显示
    uint8_t i=0;
    OLED_ShowString(4,1,"i:");

    while (1)
    {
        OLED_ShowNum(1, 6, RTC_GetCounter(), 10);    //显示32位的秒计数器
        OLED_ShowNum(3, 6, RTC_GetFlagStatus(RTC_FLAG_ALR), 1);      //显示闹钟标志位
        i++;
        OLED_ShowNum(4,3,i,3);
        Delay_ms(1000);
        if(i==5)
        {
            OLED_Clear();                        //OLED清屏，模拟关闭外部所有的耗电设备，以达到极
度省电
            PWR_EnterSTANDBYMode();          //STM32进入停止模式，并等待指定的唤醒事件（WKUP上
升沿或RTC闹钟）
            /*待机模式唤醒后，程序会重头开始运行*/
        }
    }
}
```

课程小结

不同的电源模式，对应的功耗是不一样的。

随堂作业

对比睡眠模式、停机模式、待机模式，用表格整理。